

自然语言处理

王刚

统计与数据科学学院

2026 年春季学期

目录

第十二章 知识图谱

12.1 知识图谱概述

12.2 知识图谱表示与存储

12.3 知识图谱获取与构建

12.4 知识图谱推理

12.5 知识图谱问答

第十二章知识图谱

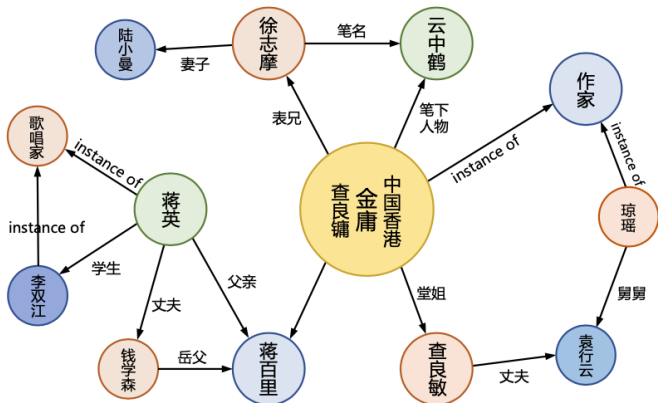
引言

- ▶ 知识图谱 (Knowledge Graph) 是谷歌公司于 2012 首次提出的概念，但是其研究历史可以追溯到费根鲍姆教授 (B.A. Feigenbaum) 在 1977 年提出的知识工程 (Knowledge Engineering) 以及 20 世纪 90 年代后期开始的语义网 (Semantic Web)。
- ▶ 知识工程、语义网以及知识图谱都属于人工智能中知识这一核心命题。知识图谱并不是单一技术的研究，而是一个系统工程，其研究内容涵盖知识表示、知识存储、知识推理、图谱构建、图谱问答等方面，还涉及自然语言处理、机器学习、图数据库、逻辑推理等多个交叉领域。
- ▶ 自然语言和知识密切关联，知识是实现计算机对自然语言真正理解必不可少的关键部分。知识图谱在自然语言处理中也发挥着越来越重要的作用，是智能问答、语义检索、机器翻译、语义表示等任务的重要基础。

12.1 知识图谱概述

知识图谱概述

- 知识图谱 (Knowledge Graph, KG) 是指采用图结构表示实体 (包括物体、事件或抽象概念) 及其之间关系的知识库 (Knowledge Base)。在知识图谱中, 图的节点表示实体, 图的边表示实体之间的关系。



知识图谱概述

- 2012 年谷歌公司在提出知识图谱概念的同时，发布了其知识搜索产品。目前绝大部分搜索引擎都将知识图谱作为重要的底层支撑技术，对于很多用户查询，也采用了基于知识图谱的问答算法直接回答用户问题。
针对“金庸和徐志摩是什么关系？”的问题，搜索引擎直接通过知识图谱给出了“表兄弟”关系，并通过生日计算得到了他们的年纪差。这种基于知识图谱的问答大幅度提升了用户使用搜索引擎的体验。



知识图谱概述

- ▶ 知识图谱按照其所描述的主要内容可以分为四类: 事实知识图谱, 概念知识图谱, 语言知识图谱和常识知识图谱。
 - (1) 事实知识图谱包含了现实世界中各实体之间的关系, 比如: “梅西” 出生于 “阿根廷”。
 - (2) 概念知识图谱描述概念之间的子类关系, 比如: “老虎” 是 “哺乳动物”, “足球运动员” 属于 “运动员”。
 - (3) 语言知识图谱是人类语言中蕴含的词法、句法、语义和语用等知识, 比如: WordNet、HowNet 都可以认为是一种词法知识图谱。
 - (4) 常识知识图谱描述人类与世界交互积累的经验与知识, 比如: “猫爱吃鱼”、“冬天可能会下雪”。

知识图谱概述

- ▶ 知识图谱一般由 < 头实体, 关系, 尾实体 > 组成的三元组为基本元素构成。实体一般为世界上的具象事物或者抽象概念, 而实体之间的联系则定义为关系。通过扩展这样的三元组, 最终会形成一张巨大的知识网络, 网络的节点为实体, 边则为实体之间的关系, 这样的知识网络即为知识图谱。知识图谱可以形式化的定义为:

$$\mathcal{G} = \{\mathcal{E}, \mathcal{R}, \mathcal{F}\}$$

其中 $\mathcal{E}$ 表示为实体集合, $\mathcal{R}$ 表示为关系集合, 而 $\mathcal{F}$ 表示为事实集合。知识图谱中的一个事实即为上文中提到的三元组 $\langle h, r, t \rangle \in \mathcal{F}$, h 表示头实体, r 表示关系, t 表示尾实体, $h, t \in \mathcal{E}, r \in \mathcal{R}$ 。

知识图谱发展历程

- ▶ 20 世纪 80 年代开始哲学领域的本体 (Ontology) 概念引入人工智能，是指用规范化方法描述概念、实体、术语及其相互关系。研究人员们也提出了语义网络 (Semantic Network) 用于形式化地表示知识。语义网使用有向或者无向图中的顶点表示概念，边表示概念之间的语义关系。
- ▶ 1993 年 Gruber 关于本体给出了更通用定义，将本体定义为对某一智能体或智能体群体中存在的概念和关系的一种描述。在这期间，构造了包括 WordNet、HowNet 等在内的大量通用和领域本体库。
- ▶ 1998 年提出了语义网 (Semantic Web) 概念，其核心是通过定义标准的标志语言 (Markup Language) 和构造相关处理工具，对互联网上的文档添加能够被计算机所理解的元数据 (Meta Data)，从而扩展互联网，使之成为一个通用的信息交换媒介。

知识图谱发展历程

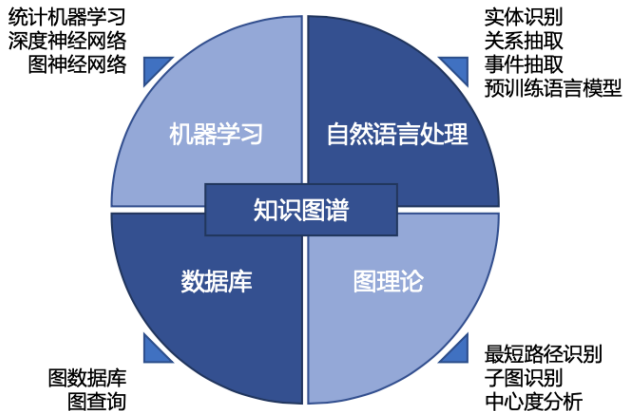
- ▶ 2012 年谷歌公司以 “Things, Not Strings!” 为主题发布了知识图谱搜索产品，希望解决用户使用传统搜索引擎需要阅读文章并自行寻找答案，即字符串级别搜索。试图构建事物 (Things) 对象级别搜索，通过构建大规模结构化的事物精确描述以及事物之间的关联，使得用户可以直接得到答案或对象级精准搜索。
- ▶ 百度在 2020 年发布的多源异构中文知识图谱，覆盖超过 50 亿实体和 5500 亿事实。阿里巴巴构建的数字商业知识图谱 AliOpenKG，包含 1600 万实体、67 万的核心概念、2681 类关系以及 18 亿三元组。美团构建的餐饮娱乐知识图谱包含 23 类概念、16 亿实体、486 亿三元组。谷歌的知识图谱也从 2012 年发布时的 5 亿实体，35 亿关系，增长到 2020 年包含 50 亿实体，5000 亿关系。知识图谱已经成为了搜索引擎、推荐系统、智能问答等应用系统中不可获取的基础组件。

知识图谱研究内容

- ▶ 1. 知识图谱表示: 知识图谱根据使用场景和应用需求, 可以采用有向图、有向标记图、本体描述语言等方式在逻辑层面进行表示。
- ▶ 2. 知识图谱存储: 随着知识图谱规模不断扩大, 如何能够高效存储和检索包含百亿级顶点千亿级边规模的图是知识图谱应用必不可少的基础工作。
- ▶ 3. 知识图谱构建: 知识广泛存在于自然语言文本、半结构化以及结构化的数据中, 知识图谱构建主要目标就是研究如何利用实体识别、关系抽取等信息抽取技术, 从自然语言文本中抽取实体、属性、关系、事件等知识图谱要素的方法, 以及属性补全、实体链接、实体对齐等知识图谱扩展和融合方法。
- ▶ 4. 知识图谱推理: 知识图谱不仅可以提供知识的存储和检索, 更重要的是可以根据构建的已知知识进行归纳、推断和预测未知的事实。
- ▶ 5. 知识图谱应用: 构建大规模知识图谱后, 如何将知识与不同的自然语言处理任务进行深度融合, 构建基于知识图谱的智能推荐系统、基于知识图谱的智能问答以及知识增强的自然语言处理算法是知识图谱应用所重点研究的内容。

知识图谱研究内容

► 知识图谱研究涉及领域



12.2 知识图谱表示与存储

知识图谱表示与存储

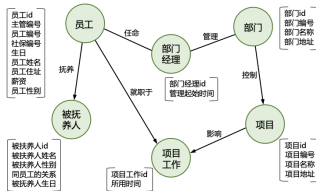
- ▶ 知识图谱应用需要使用一种合理的知识表达方式，保证结构化的知识可以被计算机正确处理。从某种意义上可以说，知识表示是贯穿知识库的构建与应用全过程的关键问题。
- ▶ 传统的知识图谱表示方法一般以符号式为主，包括属性图、RDF、OWL 本体语言等。符号表示方法的特点是可解释性强，但是依赖知识描述的准确性。
- ▶ 近年来，随着深度神经网络研究的兴起，基于向量式的表示方法逐渐引起人们重视。向量式的方法容易捕获隐式知识，计算效率高，但是可解释性差。此外，向量式表示本质上属于统计模型，对没见过的实体表示质量较差，而自然界的实体一般呈长尾分布，这也在一定程度上限制了向量式表示方法的应用。

知识图谱的符号表示

- ▶ 在简单的应用场景下，无向图即可满足需求。
- ▶ 若要进一步增强知识图谱的表达能力，给实体和边添加属性，则可以选择有向标记图。常用的有向图模型有两种：属性图和 RDF 图模型。
- ▶ 在更加复杂的场景下，比如建模传递关系、自反关系，有向标记图仍然不能满足需要，这时候可以选用 RDFS/OWL 本体语言作为描述语言对 RDF 进行扩展。

知识图谱的符号表示—属性图

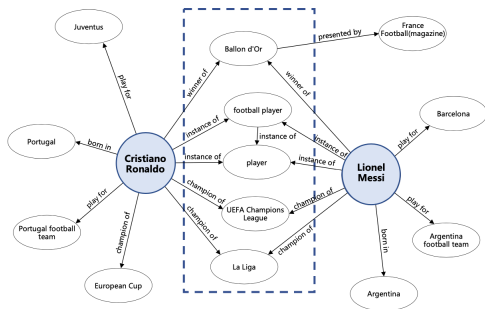
- 属性图 (Property Graph) 是一种有向标记图，包含三个要素: 节点 (Vertex)、边 (Edge)、属性 (Property)。节点对应知识图谱中的实体，边则是实体对之间的关系描述，相应的边的类型则对应实体之间的关系标签。属性本质上是一个键值对，属性图可以灵活地为节点和边添加任意属性。



- 属性图与无向图表示的不同之处在于，在属性图的模型中，关系被提升到了与实体一样重要的程度。属性图提供了一个丰富的视角，即不同种类的数据如何相关，而这些数据依赖关系在普通的关系型数据库中难以直接展示。属性图的缺点则是无法建模深层次的语义信息，这直接限制了其语义推理能力。

知识图谱的符号表示—RDF 图模型

- ▶ 资源描述框架 (Resource Description Framework, RDF), 是由国际万维网联盟 W3C 制定, 用于描述实体/资源的数据模型。RDF 的基本组成单元为一个 SPO 三元组 < 主体 (Subject), 谓词 (Predicate), 客体 (Object)>, 用来表示一条客观世界的逻辑描述或客观事实。知识图谱正是由一些相互连接的实体和属性构成, 即一条三元组就对应了知识图谱中的一条知识, 例如:< 梅西, 是, 足球运动员 >。



知识图谱的符号表示—RDF 图模型

- ▶ RDF 也存在一些问题，首先就是无法区分概念和对象。概念和对象可以类比于面向对象程序语言中的 Class 和 Object，“梅西”描述的是一个对象，而“足球运动员”描述的是一个概念，而 `<足球运动员, 是, 运动员>` 描述的是 subClassOf 关系。如果不能正确区分概念和对象，不仅会引起理解上的困难，还会给后续的知识图谱融合以及下游任务的应用带来障碍。
- ▶ 鉴于 RDF 表达能力有限，无法区分概念和对象，也无法描述类之间的关系和属性。W3C 提出了资源描述框架模式 (Resource Description Framework Schema, RDFS) 语言用来描述 RDF。RDFS 中定义了 Class 用来描述类，Domain 表示属性属于何种类别，Range 用来限制属性的取值，subClassOf 用作描述类的父类，subProperty 则描述属性的父属性。基于这些基础的表达构件，RDFS 可以实现一些简单的符号推理。
例如，基于“梅西 rdf:type 足球运动员”，“足球运动员 rdfs:subClassOf 运动员”，可以推断出“梅西 rdf:type 运动员”。

知识图谱的符号表示—OWL 本体语言

- ▶ 网络本体语言 (Web Ontology Language, OWL) 本质上是一些预定义的表达构件集合，都是用来描述 RDF 数据。但是，OWL 本体语言添加了额外的定义词汇，因此可以当做是 RDFS 的扩展版。
- ▶ OWL 本体语言引入了本体映射表达构件，
使用 `owl:equivalentClass` 表示两个类是相同的，
例如，可以定义“公司”和“企业”是相同的概念；
使用 `owl:equivalentProperty` 表示两个属性是相同的，
例如，可以定义“年龄”和“春秋”为含义相同的属性；
使用 `owl:sameAs` 表示两个实体是同一实体，
例如，“利昂内尔·梅西”和“梅西”指代同一个足球运动员。
- ▶ OWL 本体语言中也引入了关于属性的复杂描述词汇，
使用 `owl:TransitiveProperty` 表示属性具有传递性质，
例如，“属于”是具有传递性的属性，若 A 属于 B，B 属于 C，则 A 属于 C；
使用 `owl:SymmetricProperty` 表示属性具有对称性，
例如，“夫妻”是对称性的属性，若 A 与 B 是夫妻，那么 B 与 A 是夫妻；
使用 `owl:inverseOf` 定义两个属性的相反关系，
例如，“咨询”和“指导”是相反关系，若 A 咨询 B，则 B 指导 A。

知识图谱的向量表示

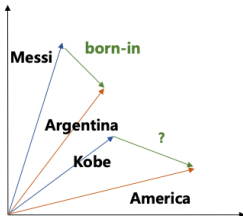
- ▶ 基于符号表示的大规模知识图谱在实际应用中面临两个问题:
 - (1) 知识表达能力差，因为知识图谱中的实体一般符合长尾分布，许多实体仅有少数关系相连，对于这些稀疏的实体与关系，很难做到充分、完整的知识表达；
 - (2) 计算效率低，基于图结构的知识表示方便人类理解，但是将其应用于下游任务时需要设计相应的图算法，这些算法计算复杂度往往较高，使得大规模知识图谱的应用门槛大大提高。
- ▶ 知识表示学习 (Knowledge Representation Learning)，即如何构建高质量的向量表示。知识表示学习通过将三元组中的语义信息投影到稠密的低维向量空间，构造实体和关系的分布式表示向量。

知识图谱向量表示学习 TransE 算法

- 词向量空间存在平移不变现象，例如：

$$v(\text{man}) - v(\text{woman}) = v(\text{king}) - v(\text{queen})$$

- TransE 模型用于学习实体和关系的向量表示。TransE 模型将关系看做是表示空间中的平移 (Translation)。在表示空间中，对于三元组 $\langle h, l, t \rangle$ ，头实体 h 的向量加上关系 r 的向量等于尾实体 t 的向量。



(Messi , born-in , Argentina)

$$h + r = t$$

(Kobe , born-in , America)

知识图谱向量表示学习 TransE 算法

- ▶ 对于给定的由三元组集合 S 组成的知识图谱，其中三元组 $\langle h, \ell, t \rangle$ 中 h 和 t 是头实体和尾实体， ℓ 是关系， $h, \ell, t \in \mathbb{R}^k$ 是对应的向量表示。
根据平移假设，对于图谱中存在的三元组应该符合 $h + \ell = t$ 。
为了保证不同三元组之间的区分度，TransE 除了知识库中存在的三元组 $(h + \ell, t)$ 之外，还构造了知识库中不存在的三元组 $(h' + \ell, t')$ ，采用 Hinge Loss 作为模型的损失函数：

$$\mathcal{L} = \sum_{\langle h, \ell, t \rangle \in S} \sum_{\langle h', \ell, t' \rangle \in S'_{\langle h, \ell, t \rangle}} [\gamma + d(h + \ell, t) - d(h' + \ell, t')]_+$$

其中 γ 为正负例的得分间隔距离， S 为正例集合， S' 为负例集合。

知识图谱向量表示学习 TransR 算法

- ▶ TransE 算法不能正确建模知识图谱中存在的一对多、多对一以及多对多的复杂关系。例如，知识图谱中存在 $\langle \text{曹操}, \text{出生于}, \text{东汉末年} \rangle$ 和 $\langle \text{刘备}, \text{出生于}, \text{东汉末年} \rangle$ 两个事实，根据通过 TransE 算法所训练出来的“曹操”和“刘备”的表示很相似，无法区分二者。又比如“曹操”在不同语境下可能是“军事家”，也可能是“诗人”。
- ▶ 针对上述问题，TransR 算法为每种关系 ℓ 定义了单独地语义空间。给定三元组 $\langle h, \ell, t \rangle$ ，头尾实体表示分别是 $h, t \in \mathbb{R}^k$ ，关系表示向量 $\ell \in \mathbb{R}^d$ ，实体表示和关系表示的维度不需要相等。

TransR 算法通过针对每种关系的不同的映射矩阵 M_ℓ 将实体映射到关系空间中，要求实体在关系空间中满足平移关系即可。特定关系投影可以使实际具有该关系的头/尾实体彼此靠近，也可以使得不具有该关系的实体则彼此远离。

TransR 本质上是放宽了 TransE 的 $h + \ell = t$ 假设，将原来在一个空间中满足的平移关系，改为在不同的关系空间中满足。

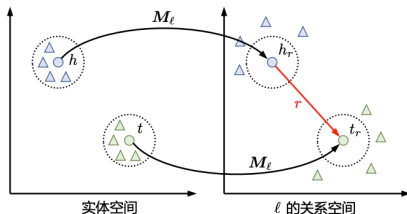
知识图谱向量表示学习 TransR 算法

- 给定三元组 $\langle h, \ell, t \rangle$, TransR 算法通过关系 ℓ 对应的映射矩阵 M_ℓ , 将头实体和尾实体映射到关系空间中, 得到实体表示 h_ℓ 和 t_ℓ :

$$h_\ell = hM_\ell, \quad t_\ell = tM_\ell$$

在得到对应关系空间的表示 h_ℓ 和 t_ℓ 后, 就可以使用与 TransE 类似的评分函数进行训练:

$$f_\ell(h, t) = ||h_\ell + \ell - t_\ell||_2^2$$



基于表的知识图谱存储

- ▶ 尽管知识图谱是用图的形式描述，但图数据库并不是存储知识图谱的唯一方案。在工业界，很多成熟的数据库都是基于关系模型，知识图谱的数据可以通过一定的设计，从而可以利用关系型数据库存储。基于关系型数据库的知识图谱存储方案主要可以分为四种：
 - 基于三元组的知识图谱存储、
 - 基于属性表的知识图谱存储、
 - 基于垂直表的知识图谱存储、
 - 基于全索引的知识图谱存储。

基于三元组的知识图谱存储

- ▶ 利用关系型数据存储知识图谱最简单的办法就是构建一个由主体、谓词和客体三列构成的表，将知识图谱中的每个 SPO 三元组看做该表中的一条记录，直接存储到关系型数据库中。
- ▶ 当用户输入一个查询请求时，需要将其转换为对应的查询 SQL 语句。但是稍微复杂一点的查询，需要大量的自连接 (Self-Join) 操作。例如，用户需要查询“获得金球奖的足球运动员出生地”，这需要将整张表复制三次，然后依次扫描，基于查询结果再做连接 (Join) 操作，查询代价过于昂贵。

主体	谓词	客体
利昂内尔·梅西	出生地	阿根廷
利昂内尔·梅西	isA	足球运动员
利昂内尔·梅西	生日	1987-06-24
利昂内尔·梅西	效力	巴塞罗那足球俱乐部
利昂内尔·梅西	荣誉	金球奖
杰西·林加德	出生地	英格兰
杰西·林加德	isA	足球运动员
杰西·林加德	效力	曼彻斯特联足球俱乐部
科比·布莱恩特	出生地	美国
科比·布莱恩特	isA	篮球运动员
科比·布莱恩特	效力	洛杉矶湖人队
科比·布莱恩特	荣誉	NBA 总决赛
足球运动员	subClassOf	运动员
篮球运动员	subClassOf	运动员
金球奖	英文名	Ballon d'Or
NBA 总决赛 MVP	英文名	NBA Finals MVP

基于属性表的知识图谱存储

- 为了解决基于三元组的知识图谱存储查询效率过低问题，还可以按照属性表 (Property Tables) 的方式存储知识图谱。其存储方案仍然是基于关系型数据库，不同之处在于，属性表根据实体类型对三元组进行分类，不同类别的三元组存储于不同的表中。例如，针对上一节中的三元组存储表案例转换为属性表后，其结构如表所示。将运动员、类型、生日、出生地、荣誉等信息合并到一张表中进行保存。对于上例中用户查询“获得金球奖的足球运动员出生地”，只需要检索“荣誉”对应的属性表就可以完成。
- 知识图谱的数据往往具备稀疏性，因此基于属性表的存储方案可能会包含大量空值，造成存储空间的极大浪费。此外，属性表的存储方案还依赖于人的设计，哪些属性需要合并在一张表中，需要根据知识图谱需求和实际关系联合考虑。

主体	isA	生日	出生地	效力	荣誉
利昂内尔·梅西	足球运动员	1987-06-24	阿根廷	巴黎圣日耳曼足球俱乐部	金球奖
杰西·林加德	足球运动员		英格兰	曼彻斯特联足球俱乐部	
科比·布莱恩特	篮球运动员		美国	洛杉矶湖人队	NBA 总决赛 MVP

主体	英文名
金球奖	Ballon d'Or
NBA 总决赛 MVP	NBA Finals MVP

主体	subClassOf
足球运动员	运动员
篮球运动员	运动员

基于垂直表的知识图谱存储

- 除了按照实体对三元组表格分类外，还可以选择按照谓词构造表格，这种方式称为基于垂直表 (Vertical Tables) 的存储方式。将不同的谓词对应不同的表格，这种方法不仅可以避免查询过程中大量的自连接操作，还不会在数据库中产生大量的空值。根据谓词构建了 isA、出生地、效力等多个表。
- 这种方法缺点是对于具备多个属性的实体，往往需要对多个表进行插入操作，操作效率低。对于属性不确定的查询，垂直表也不能很好的处理，例如，查询“科比·布莱恩特和洛杉矶湖人队的关系”，就需要检索所有表才能得到结果，这类查询的检索性能较差。

isA

主体	客体
利昂内尔·梅西	足球运动员
杰西·林加德	足球运动员
科比·布莱恩特	篮球运动员

效力

主体	客体
利昂内尔·梅西	巴黎圣日耳曼足球俱乐部
杰西·林加德	曼彻斯特联足球俱乐部
科比·布莱恩特	洛杉矶湖人队

英文名

主体	客体
金球奖	Ballon d'Or
NBA 总决赛 MVP	NBA Finals MVP

出生地

主体	客体
利昂内尔·梅西	阿根廷
科比·布莱恩特	美国
杰西·林加德	英格兰

荣誉

主体	客体
利昂内尔·梅西	金球奖
科比·布莱恩特	NBA 总决赛 MVP

生日

主体	客体
利昂内尔·梅西	1987-06-24

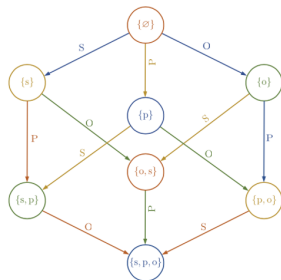
subClassOf

主体	客体
足球运动员	运动员
篮球运动员	运动员

基于全索引的知识图谱存储

- 基于全索引的存储方式通过对关系型数据库建立多种索引的方式，达到查询效率优化的目的。首先会将 SPO 三元组里的每个元素集合映射为一个数字 ID 集合，通过存储 ID 而不是字符串可以节省大量存储空间。然后，对三元组中主体、谓词、客体的各种排列情况都分别建立索引，这种组合一共有六种:SPO、POS、PSO、OSP、OPS、SOP。无论是基于谓词查询主体和客体，还是基于客体查询谓词和主体，六种索引都可以满足需求。全索引的方法应对简单的查询效率高，但是其索引的更新与维护同样代价高昂。

Value	ID	Value	ID
利昂内尔·梅西	0	1987-06-24	12
杰西·林加德	1	巴黎圣日耳曼足球俱乐部	13
科比·布莱恩特	2	金球奖	14
出生地	3	英格兰	15
isA	4	曼彻斯特联足球俱乐部	16
生日	5	美国	17
效力	6	篮球运动员	18
荣誉	7	洛杉矶湖人队	19
subClassof	8	NBA 总决赛 MVP	20
英文名	9	运动员	21
阿根廷	10	Ballon d'Or	22
足球运动员	11	NBA Finals MVP	23



基于图的知识图谱存储

- ▶ 图数据库则充分利用图的结构对数据进行建模，将一张图对应到一个邻接列表，再基于这个邻接列表建立索引，优化关联查询。

例如，查询“获得 NBA Finals MVP 荣誉的球员效力于哪个俱乐部”，只要依次对“NBA Finals MVP”、“NBA 总决赛 MVP”、“科比·布莱恩特”三个节点的邻居节点进行检索即可。

- ▶ 在图数据库中，关系是被显式描述和刻画的，属性也可以单独定义，这就极大地增加了数据建模的灵活性。随着相关技术和工具的逐渐成熟，图数据库已逐渐成为知识图谱存储和查询引擎搭建的基础设施。

	基于属性图的图数据库	RDF 存储系统	关系型数据库
数据模型	属性图	RDF 三元组	关系数据模型
查询语言	Cypher、Gremlin ^[652]	SPARQL	SQL
应用场景	多为工业界	多为学术界	工业界和学术界均有使用
其他特点	图遍历效率高；图的全局操作效率低	有标准的推理引擎；易于数据发布	简单查询销量高；多跳查询会产生连接操作，效率低

图：知识图谱不同存储方案比较

12.3 知识图谱获取与构建

知识图谱获取与构建

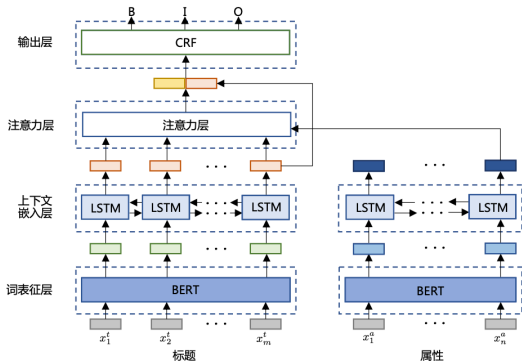
- ▶ 知识图谱的数据来源是多种多样的，包括结构化数据、半结构化数据以及非结构化数据等。
 - a) 对于已有的结构化数据，可以通过一定的规则来提取其中的知识，比如关系型数据库，其行 (Row) 可能代表知识图谱中的实例 (Instance)，列 (Column) 可能表示属性 (Property)，单元 (Cell) 对应属性值，外键 (Foreign Key) 则可以表示指代关系。
 - b) 对于半结构化数据，比如维基百科，可以制定解析规则，将页面上的字段映射为知识图谱中的实体或者属性。
 - c) 对于非结构化数据，即纯文本数据，首先需要从文本中挖掘出目标知识图谱所需的实体，可以应用命名实体抽取等技术；基于已知实体信息，还需要从文本中识别出它们之间的关系，这对应了 SPO 三元组中的谓词关系，需要使用关系抽取技术；除了实体和关系信息，还需要抽取属性信息，需要使用属性补全技术；更复杂的场景下需要对事件信息进行抽取，可以看作一组三元组的联合抽取过程。

属性补全

- ▶ 属性补全的目的就是自动地从文本中提取出实体所具备的属性，特别是在电子商务等垂直领域下，属性补全有着重要的应用，若能自动从产品描述中提取出产品的属性，将能更好的描述产品特征，优化产品搜索和推荐效果。
- ▶ 属性补全方法通常采用抽取式方法，类似于命名实体识别，将属性补全转为序列标注任务，即对句子中的每个字进行分类，在此基础上基于字的分类结果解析得到属性值。
- ▶ 但如果简单地将其等价于普通的序列标注任务，将属性类型等价于实体类型，会带来两个问题：
 - (1) 普通的命名实体识别模型一般只建模数个实体类型，而领域知识图谱可能包含成千上万的属性种类，且这些属性往往符合二八法则 (Power Laws)，直接应用普通的序列标注模型，由于标签数量巨大，会造成性能的显著下降；
 - (2) 知识图谱的构建需要保证拓展性，保留属性集合扩展的可能，而当前实体识别模型一般是基于封闭世界假设，无法识别新的实体种类，若直接应用，同样无法识别未标注过的新属性。

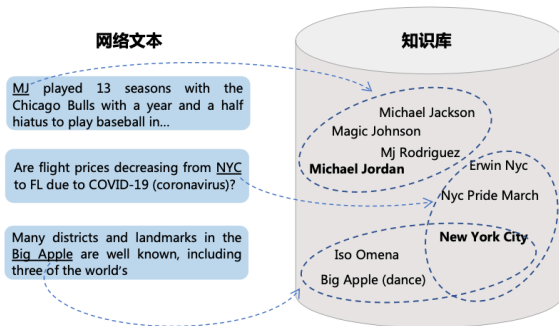
属性补全

- 为了解决这个问题，一种基于属性理解的属性抽取方法 AVEPT 将属性理解问题转化为抽取式阅读理解任务，待抽取的文本作为上下文，目标属性信息作为问句，属性值则作为答案，是模型要抽取出来的目标。由此，将抽取结果和属性类型进行了解偶，并且输入的属性信息可以帮助模型捕获属性的语义信息。因此，该方法在取得较好的效果同时，还保留了模型对新属性识别的拓展性。



实体链接

- ▶ 实体链接 (Entity Linking) 目标是将文本中的实体指代和它们在知识图谱中的对应实体进行对应。对已有知识图谱扩充的时候，也需要实体链接任务将文本中的实体指代与其知识图谱中真正对应的实体完成映射。实体链接任务也是很多下游应用的预处理工作，如问答、关系抽取、内容分析等。



实体链接

- ▶ 实体链接的任务的难点主要有两个:

(1) 实体的歧义性,

例如: “MJ” 可能表示篮球运动员 “Michael Jordan”, 也可能是美国歌手 “Mj Rodriguez”;

(2) 实体表达的多样性, 即一个命名实体有不同的表示形式, 如全称、缩写、别名等等,

例如: “NYC” 和 “Big Apple” 是命名实体 “New York City” 的缩写和昵称。

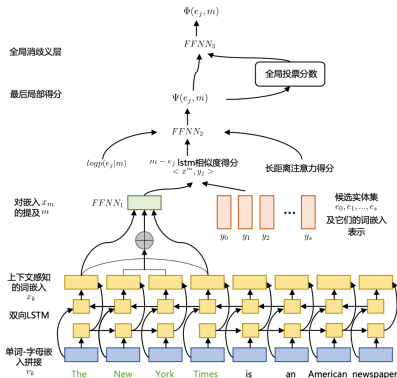
歧义性和多样性的消除需要依据上下文语义信息以及实体属性和关系信息。

实体链接

- ▶ 基于传统机器学习的实体链接算法
首先，提取人工设计的特征，如实体的 TF-IDF 值、局部上下文兼容性和文档级的实体引用全局一致性的特征等等。然后，将特征输入给实体排序模型，进行最后的链接预测。
- ▶ 方法的不足：
 - 1) 需要进行大量细致而繁琐的特征工程；
 - 2) 由于特征设计过程对特定知识库或者领域知识强烈依赖，很难将已有的实体链接方法推广到其他知识库或者领域。

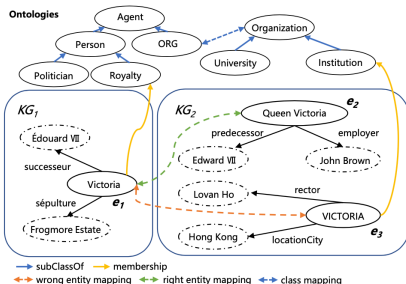
实体链接

- 深度学习算法通常将实体链接任务分解为实体识别和实体消歧两部分，对它们分别建模，但是这样会导致模型错误传递。
- ENEL(End-to-End Neural Entity Linking) 同时建模实体识别和实体消歧任务，是一种端到端的方法。主要思想是考虑所有可能的文本片段作为潜在的实体，并学习其候选实体的上下文相似性分数。



实体对齐

- ▶ 本体和实例的异构性
 - a) 不同领域的知识存在差异性，同时不同专家对知识定义的主观性，使得无法构建统一的本体知识。
 - b) 知识本身会随着时间发生演变，在不同时间，使用者会构建不同的知识本体，对相同实体的指代称呼也会发生变化。
- ▶ 知识融合技术：目标就是为不同知识图谱的实例或者对象建立关联，可以高效合并多个不同来源的知识图谱。



实体对齐

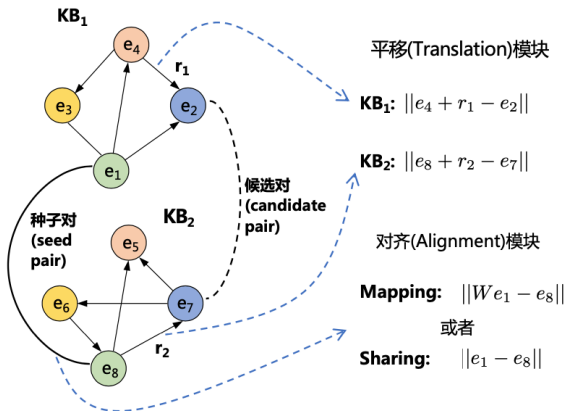
- ▶ 知识融合技术的两个基本任务
 - 1) 本体匹配：用来匹配本体层的抽象知识；
 - 2) 实体对齐：匹配实例层中相同对象的不同实体指代。两者使用的技术有很多类似，但实例规模通常远大于本体规模，任务难度更大。
- ▶ 实体对齐 (Entity Alignment) 也被称作实体匹配 (Entity Matching)，旨在发现多源知识图谱中等价的实体对。比如由几种不同的语言构建的知识库，实现跨语言知识对齐将帮助人们构建一个连的知识库，并帮助机器处理跨语言的实体关系的不同表达。
 - a) 基于实体属性的相似度计算。这种方法依赖于用户定义的规则来决定实体之间进行比较的属性。
 - b) 基于表示学习的实体对齐，核心思想就是在低维空间中用向量的距离来计算不同来源的实体之间的相似度。

基于平移的实体对齐方法

- ▶ 基于平移的知识表示可提高单语言知识的完整性，但不能直接解决跨语言知识的对齐问题。主要有以下几个方面的原因：
 - (1) 跨语言平移比任何单语关系平移的范围都要大得多；
 - (2) 实体和关系在不同语言之间的词汇表达不连贯；
 - (3) 用于训练知识嵌入表示的已知对齐关系通常只占知识库的一小部分。
- ▶ MTransE 算法通过在独立的嵌入空间中编码每种语言的实体和关系，为每个嵌入向量提供到其他语言空间中对应向量的过渡，同时保留单语言嵌入的功能。MTransE 的核心假设是相同实体在不同来源的知识图谱里存在类似的分布。

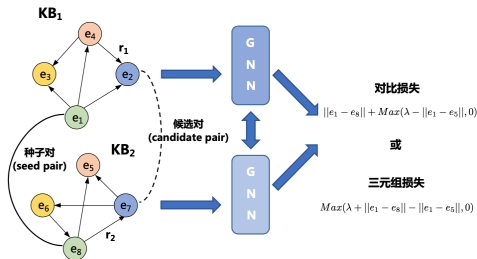
基于平移的实体对齐方法

- 平移模块在特定于语言的知识图谱中编码实体和关系，采用 TransE 作为知识嵌入模型。在此基础上，对齐模块学习跨语言的实体和跨不同嵌入空间的关系对齐。



基于图神经网络的实体对齐方法

- ▶ 基于平移的知识嵌入模型实现的实体对齐模型，需要小心调节平移模块和对齐模块的损失权重，比较难优化。而且基于平移的方法的训练目标只包含单独的三元组，而实体的属性信息并没有被有效利用，所以其并不能从全局视角捕获实体和关系的信息。
- ▶ 因此，GCNAlign 算法，引入图卷积神经网络来建模全局信息，包含两个图神经网络，分别将不同源的知识图谱实体编码到一个统一的向量空间，然后通过对比损失或者三元组损失优化它们之间的距离。
- ▶ GCNAlign 包含三部分：基于图卷积网络的实体嵌入学习、对齐实体的预测和模型训练方法。



基于图神经网络的实体对齐方法

基于图卷积网络的实体嵌入学习:

- ▶ GCNAlign 的核心假设是:
 - (1) 等价的实体一般拥有类似的属性;
 - (2) 等价的实体其邻居实体一般也等价。
- ▶ 图卷积神经网络是一种直接对图数据进行操作的神经网络, 其输入是节点的特征向量和图的结构。GCN 的目标是学习输入图上特征的函数, 并产生节点级输出。因此, GCNAlign 使用 GCN 将属性信息和结构信息结合在一起, 把实体投影到低维向量空间中, 并约束在低维向量空间中等价实体之间彼此接近。
- ▶ 为了同时利用知识图谱中的结构信息和实体的属性信息, GCNAlign 建模两种特征: 结构特征 h_s 和属性特征 h_a 。
- ▶ GCNAlign 使用了两个两层的 GCN, 分别建模两种语言的嵌入表示。知识图谱是个有向图, 且实体之间通过代表不同关系的边相连接, 因此 GCNAlign 设计了一种特定的邻接矩阵 A 的方法。

基于图神经网络的实体对齐方法

对齐实体的预测:

- ▶ 通过将预定义的距离函数应用于实体的 GCN 表示来预测实体对齐。对于来源于知识图谱 G_1 的实体 e_i 和来源于知识图谱 G_2 的实体 v_j ，两者的距离计算方式如下

$$D(e_i, v_j) = \beta \frac{f(h_s(e_i), h_s(v_j))}{d_s} + (1 - \beta) \frac{f(h_a(e_i), h_a(v_j))}{d_a}$$

其中 $f(x, y) = \|x - y\|_1$ ，而 $h_s(\cdot)$ 和 $h_a(\cdot)$ 表示实体的结构嵌入向量和属性嵌入向量， d_s 和 d_a 表示结构嵌入向量和属性嵌入向量的维度， β 则为平衡两者重要性的超参数。

- ▶ 对于对齐的实体，两者距离应该较小，对于非对齐实体，该距离预计较大。给定 G_1 中的特定实体 e_i ，计算 e_i 与 G_2 中的所有实体之间的距离，并返回排序实体列表作为候选对齐，也可以从 G_2 到 G_1 进行对齐。

基于图神经网络的实体对齐方法

模型训练:

- 为了使 GCN 在向量空间中嵌入尽可能接近的等价实体, GCNAlign 使用一组已知的实体对齐集合 S 作为训练数据来训练 GCN 模型。通过最小化以下基于边际的排序损失函数来进行模型训练:

$$\mathcal{L}_s = \sum_{(e,v) \in S} \sum_{(e',v') \in S'_{(e,v)}} [f(h_s(e), h_s(v)) + \gamma_s - f(h_s(e'), h_s(v'))]_+$$

$$\mathcal{L}_a = \sum_{(e,v) \in S} \sum_{(e',v') \in S'_{(e,v)}} [f(h_a(e), h_a(v)) + \gamma_a - f(h_a(e'), h_a(v'))]_+$$

其中 $[x]_+ = \max 0, x$, $S'_{e,v}$ 表示由非对齐实体对的负例集合, 比如将实体 e 或实体 v 替换为 G_1 或 G_2 中的一个随机实体; $\gamma_s, \gamma_a > 0$ 为分离正负例的间隔超参。 $\mathcal{L}_s$ 和 $\mathcal{L}_a$ 是结构嵌入和属性嵌入的损失函数, 它们互相独立, 可以分开优化。

12.4 知识图谱推理

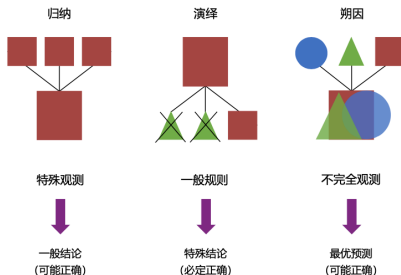
知识图谱推理

► 常见的推理方法：

a) 演绎推理 (Deductive Reasoning) 是一种自上而下的推理方法，即根据已知的一般或普遍性前提推理得到必然成立的结论。

b) 归纳推理 (Inductive Reasoning) 是一种自下而上的推理方法，即根据观察所得到客观知识进行总结归纳，从而得到更一般的结论。

c) 溯因推理 (Abductive Reasoning) 是从现象出发的推理方法，即结合已有的观察和知识，推断出有可能的解释过程。



知识图谱推理

- ▶ 基于知识图谱的知识推理 (Knowledge Reasoning), 旨在从图谱中已存在的关联关系或事实推断出未知的关系或事实。推理得到的知识又可以反过来丰富知识图谱, 为下游的应用提供更好的支持。
- ▶ 从推理方法的角度, 知识图谱推理方法可以分为两类:
 - 1) 一种是演绎推理, 主要是基于符号逻辑的知识图谱推理, 依赖于专家显示制定的知识描述和逻辑推导方法;
 - 2) 一种是归纳推理, 代表方法为基于表示学习的知识图谱推理, 更多地依赖于大规模数据和统计学习算法。

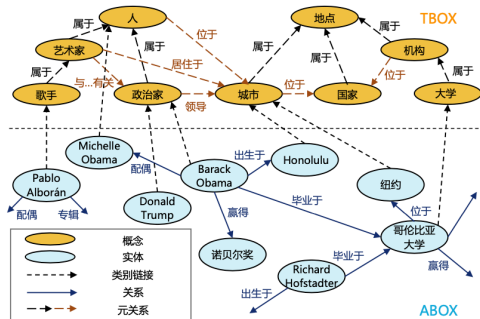
基于符号逻辑的知识图谱推理—1. 基于本体的推理

► 一个定义完备的知识图谱包含两部分知识:

(1) TBox(Theory Box), 即是关于概念术语的断言, 对应 Schema 层, 主要定义概念以及关系;

(2) ABox(Assertion Box), 是关于个体的断言, 关于事实性的描述。

基于本体的推理方法, 一般在 TBox 层定义本体公理 (Ontological Axioms), 然后根据演绎推理的方法推断出新的事实。



基于符号逻辑的知识图谱推理—1. 基于本体的推理

- ▶ OWL 在 RDFS 进一步扩展了 Schema 的表达能力，可以完成更复杂的本体逻辑推理。

构造算子	语法	语义	例子
原子概念	A	$A^I \subseteq \Delta^I$	Human
原子关系	R	$R^I \subseteq \Delta^I \times \Delta^I$	has_child
对概念 C , D 和关系 (role) R			
合取	$C \sqcap D$	$C^I \cap D^I$	Human $\sqcap$ Male
析取	$C \sqcup D$	$C^I \cup D^I$	Doctor $\sqcup$ Lawyer
非	$\neg C$	$\Delta^I \setminus C^I$	$\neg$ Male
存在量词	$\exists R.C$	$\{x \exists y. \langle x, y \rangle \in R^I \wedge y \in C^I\}$	$\exists$ has_child.Male
全称量词	$\forall R.C$	$\{x \forall y. \langle x, y \rangle \in R^I \wedge y \Rightarrow C^I\}$	$\forall$ has_child.Doctor

- ▶ 基于 OWL 的基础语法和公理基本语法，知识图谱可以实现一定的推理能力。例如: TBox 中定义“人工智能公司 subclass 高科技公司”，ABox 中存在知识“谷歌 type 人工智能公司”，那么可以推断出“谷歌 type 高科技公司”；又比如在 OWL 本体中定义“父亲的父亲是爷爷”，已知“司马懿是司马昭的父亲”和“司马昭是司马炎的父亲”，可以推断出“司马懿是司马炎的爷爷”。

基于符号逻辑的知识图谱推理—1. 基于本体的推理

- ▶ OWL 本体推理的核心算法为 Tableaux 算法，其基本方法是通过一系列规则构建 ABox，然后检查知识库的可满足性。Tableaux 是一颗公式树，它会根据前提和否定结论来不断创建分支，对公式进行逐级分解，当所有分支都关闭后，Tableaux 算法就会终止。简单来说，如果要证明一件事是正确的，只要将其所有反例驳斥即可达到目的。

$\sqcap^+$ -规则：若 $C \sqcap D(x) \in \emptyset$ ，且 $C(x), D(x) \notin \emptyset$ ，则 $\emptyset := \emptyset \cup \{C(x), D(x)\}$ ；

$\sqcap^-$ -规则：若 $C(x), D(x) \in \emptyset$ ，且 $C(x) \sqcap D(x) \notin \emptyset$ ，则 $\emptyset := \emptyset \cup \{C(x), D(x)\}$ ；

$\exists$ -规则：若 $\exists R.C(x) \in \emptyset$ ，且 $R(x, y), C(y) \notin \emptyset$ ，则 $\emptyset := \emptyset \cup \{R(x, y), C(y)\}$ ，其中， y 是新加进来的个体；

$\forall$ -规则：若 $\forall R.C(x), R(x, y) \in \emptyset$ ，且 $C(y) \notin \emptyset$ ，则 $\emptyset := \emptyset \cup \{C(y)\}$ ；

$\sqsubseteq$ -规则：若 $C(x) \in \emptyset, C \sqsubseteq D$ ，且 $D(x) \notin \emptyset$ ，则 $\emptyset := \emptyset \cup \{D(x)\}$ ；

$\perp$ -规则：若 $\perp(x) \in \emptyset$ ，则拒绝 $\emptyset$ ；

- ▶ 示例：第一条规则，如果 ABox 中声明 x 属于 C 和 D 的组合类，但是 $C(x)$ 和 $D(x)$ 并不在 ABox 中，则把 $C(x)$ 和 $D(x)$ 都加入到 ABox 中。

基于符号逻辑的知识图谱推理—2. 基于 Datalog 推理

- ▶ 基于本体的知识推理只能基于本体概念描述推理，不支持规则型知识的推理。此外，用户无法定义自己的推理过程。Datalog 则用于解决逻辑问题，将本体推理和规则推理相结合。
- ▶ Datalog 是一种基于逻辑编程语言 Prolog 且适应于知识库的改进型语言，能够方便地和大型数据库进行交互，便于撰写规则实现推理。

基于符号逻辑的知识图谱推理—2. 基于 Datalog 推理

- ▶ Datalog 的基本组成是原子 $p(t_1, t_2, \dots, t_n)$ ，其中 p 是谓词， t_i 是变量或者常量。
例如: $\text{has_parent}(X, Y)$ ，其中 “has_parent” 是谓词，“X, Y” 是常量。
一条规则 $H : B_1, B_2, \dots, B_m$ 由一个头部原子 H 和多个体部原子 $B_1, B_2, \dots, B_m$ 构成，表示当体部原子成立时，可以得到头部原子成立的结论。

例如： $\text{has_parent}(X, Y) : \text{has_mother}(X, Y)$

表示 “has_mother(X, Y)” 蕴含了 “has_parent(X, Y)”

- ▶ Datalog 程序本质上就是规则的集合，
例如: 已知规则集合 “坐落于 (X, Y):-同城 (X, Z), 坐落于 (Z, Y)”，和事实集合 “同城 (复旦大学, 华东师范大学):-; 坐落于 (华东师范大学, 上海):-”，基于第一条规则，第一条和第二条事实，可以推理出新的事实 “坐落于 (复旦大学, 上海)”。在实际应用中，事实集合可能非常大，随着规则集合越来越大，推理的开销也会显著升高。

基于符号逻辑的知识图谱推理—3. 基于产生式规则

- ▶ 产生式规则推理方法是一种前向推理系统，从已知事实出发，通过一定规则求得结论，类似于一阶逻辑。其基本组成包含:Working Memory(WM) 中的事实集合，产生式规则集合以及推理引擎。WM 中存储的事实数据包括两种，一是用来描述对象，例如:type attr1:val1 attr2:val2 ... attrn:valn，其 type, attri, vali 均为原子；另一种用来描述关系，例如:basicFact relation: olderThan firstArg:John secondArg:Alice。
- ▶ 产生式规则集合中存储的规则定义为“IF conditions THAN actions”形式，“conditions”对应条件集合，又称为 LHS(Left Hand Side)，“actions”对应动作序列，又称为 RHS(Right Hand Side)。当 LHS 中所有条件都被满足的时候，那么该规则触发，执行相应的动作。
例如:IF (Gauge state: OK) AND (TEMPERATURE > 120)
THEN (Cooling system state: over-heating)
表示: 当测量器状态正常且测量温度超过 120 度，那么制冷系统的工作状态为过载

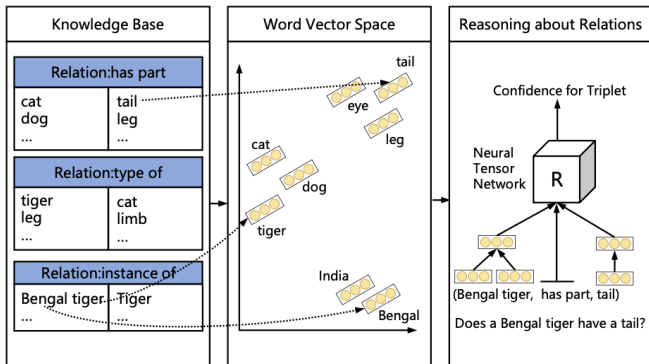
基于符号逻辑的知识图谱推理—3. 基于产生式规则

- ▶ 产生式规则的执行由推理引擎控制，其核心任务是模式匹配。这个过程参考关系型数据库做知识图谱的存储和查询，不同的条件匹配会产生大量的连接操作，是个组合爆炸问题。此外，还要考虑当满足多个触发规则时，选择哪条规则执行的问题，这被称为冲突解决。在选定规则时，则执行规则的 actions，对应 WM 中事实数据的增删改。
- ▶ Rete 算法是产生式规则系统中常用的推理算法，其核心思路是逐渐扩充条件集合，同时缓存条件匹配的中间结果，直到所有规则对应的条件集合都得到判断，是一种空间换时间的优化方法。

基于表示学习的知识图谱推理

基于神经张量网络的知识图谱推理算法

- ▶ 神经张量网络 (Neural Tensor Network, NTN) 分类实体对之间的关系，并使用构成实体的词向量的平均作为该实体表示。



基于表示学习的知识图谱推理

基于神经张量网络的知识图谱推理算法

- ▶ NTN 算法的核心是关系分类模块，该模块用来判断实体对 (e_1, e_2) 是否存在某种特定的关系 R 。假设知识库中仅建模两种关系，不同于以往的方法直接将实体对的表示拼接在一起，NTN 采用双线性神经层 W_R 直接建模实体对之间的语义关系。
- ▶ 使用双线性神经层的好处是，它可以使用乘法直接关联两个实体表示，而不是像标准神经网络那样隐含地关联实体向量。直观地说，张量 W_R 的每个切片负责一种类型的实体对或关系的实例化。例如，模型可以从词向量空间中学习到组成关系，比如 $(\text{dog}, \text{haspart}, \text{leg})$ 和 $(\text{car}, \text{haspart}, \text{engine})$ 。
- ▶ 模型的训练，即真实存在的三元组 $T^{(i)} = \langle e_1^{(i)}, R^{(i)}, e_2^{(i)} \rangle$ 相比随机实体替换得到的三元组 $T_c^{(i)} = \langle e_1^{(i)}, R^{(i)}, e_c \rangle$ 能够得到更高的分数。

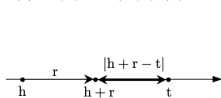
$$\mathcal{L}(\Omega) = \sum_{i=1}^N \sum_{c=1}^C \max \left(0, 1 - g \left(T^{(i)} \right) + g \left(T_c^{(i)} \right) \right) + \lambda \|\Omega\|_2^2$$

其中 N 为训练集中三元组的个数，正确的三元组打分与替换三元组打分之间的差值最多为 1，为每个三元组生成 C 个替换后的三元组。

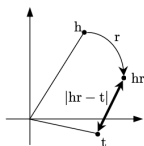
基于表示学习的知识图谱推理

基于复空间关系旋转的知识图谱推理

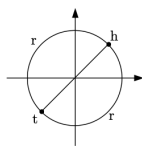
- ▶ 此前的方法只是显式或是隐式地建模部分 (对称/反对称关系、反转关系、组合关系) 关系，无法做到对三种类型关系的准确推断。
- ▶ RotatE 是受到欧拉公式的启发，即 $e^{i\theta} = \cos \theta + i \sin \theta$ ，一个酉复数可以看作是在复平面上的一个旋转。RotatE 将实体和关系映射到复向量空间中，然后将关系看作是从头实体向量往尾实体向量的旋转。



(a) TransE模型通过实线来建模关系r



(b) RotatE模型通过复平面来建模关系r



(c) RotatE模型通过指定 $r_i = -1$ 来建模对称关系r

TransE 将关系向量建模为实线上的向量平移，可以建模反转关系、反对称关系和组合关系，但是无法建模对称关系。但是 RotatE 将关系向量表示为复平面上的旋转，除 TransE 可建模的关系之外，令关系向量满足 $r_i = -1$ ，使其满足对称关系的性质。

12.5 知识图谱问答

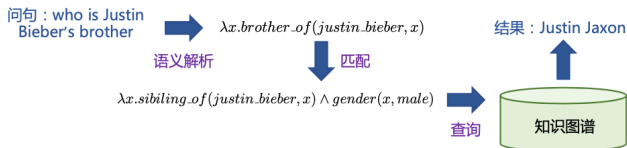
知识图谱问答

- ▶ 知识图谱问答旨在以知识库为知识源来回答自然语言问题。与文本问答相比，知识图谱的结构化知识能够提供更加精准的答案，并且方便扩展。知识图谱问答在许多智能应用中发挥着重要作用。
- ▶ 知识图谱问答方法大概可以分为两类：
 - (1) 基于语义解析的方法，即将自然语言组织的问句转化为知识库可以识别的结构化查询语句，然后在知识库中查询得到答案。而这个问句到结构化查询的映射，可以通过规则定义的模板或者训练过的自然语言解析器来完成；
 - (2) 基于信息检索的方法，通过对问句中的实体和关系识别锁定问题的主题实体，然后根据主题实体得到知识图谱中的候选实体，最后对候选进行排序得到最终答案。

基于语义解析的知识图谱问答

- ▶ 直接将问句转化为语义表示的方法存在两个问题: 一是非结构化的自然语言中许多的实体或是关系名称并不能直接匹配; 此外, 自然语言是多样化和口语化的, 模板和知识库无法覆盖所有描述, 难以将所有自然语言问句进行转化。
- ▶ 另外一种方式采用两步解析, 首先将自然语言问句转化成一种中间表示, 再进一步将中间表示翻译成逻辑语言, 得到查询语句或查询语句的逻辑形式, 在知识库上执行所得查询语句获得正确答案集。

如图示例。首先将问句转换为了与自然语言更加接近的中间表示。同样是针对问句“who is Justin Bieber's brother?”, 中间表示 `brother_of` 与句子中的单词直接可以匹配。

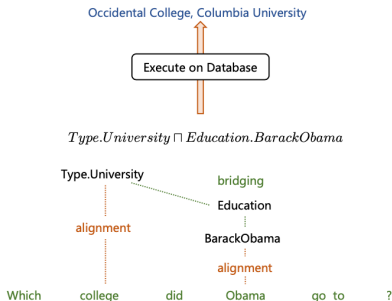


基于语义解析的知识图谱问答—1. 逻辑表达语言

- ▶ 有很多种逻辑表达语言，例如 Lamda 算子经常被用来对问句进行形式化描述。
- ▶ Lamda 演算子由一个单一转换规则和一个单一函数定义系统构成。包括函数定义、标识符引用和函数应用三个部分。对于问题 “What states border Texax?”，定义 x 是一个变量， λx 表示获取一个参数 x 并且等待返回值的 lambda 函数，接下来通过两个函数 state 和 border 代表类型以及谓词语义，最后将函数 state 和 border 应用于参数 x 。此外，可以添加其他修饰符，如存在量词、最大最小等，可进一步增强语义刻画能力。
据此上述问句可以转换为 $\lambda x.\text{state}(x) \wedge \text{borders}(x, \text{texas})$ 。

基于语义解析的知识图谱问答—1. 逻辑表达语言

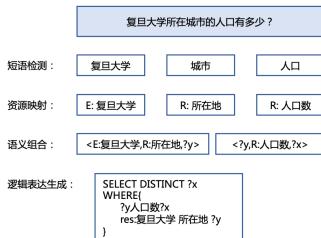
- 依存组合语义 (Dependency-based Compositional Semantics, DCS) 是一种特殊的操作桥接操作，把两个独立的语义表示片段连接起来。在语义分析过程，句子中汇总了不同片段对应的语义表示，可能无法直接进行合并，但是很多情况下过于分散的语义片段无法匹配到知识库中内容。引入桥接操作可以将两个不相邻的语义相连接，从而达到解析复杂语义的目的，尽可能连接这些离散的语义片段，在一定程度上保证语义分析能够完整。



基于语义解析的知识图谱问答—2. 语义解析

► 语义解析步骤:

- 1) 短语检索: 用于识别出问句中的实体、关系等各种短语。
- 2) 资源映射: 目标是建立问句和知识图谱之间的映射, 这包括实体链接、概念匹配和关系分类。
- 3) 语义组合: 可以使用机器学习或者深度学习方法, 生成有效的逻辑表达式。
- 4) 逻辑表达式生成: 根据得到语义组合构建逻辑表达式, 并在此基础上, 结合知识图谱的存储结构, 生成检索语句。利用检索语句从知识库中检索得到的结果, 就可以用于回答问题。

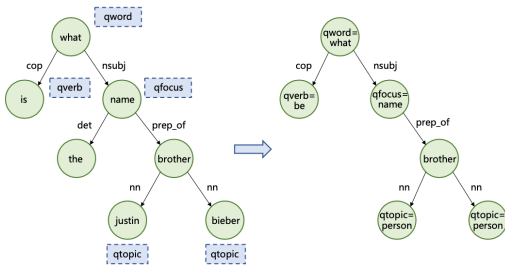


基于信息检索的知识图谱问答

- ▶ 结合知识库来回答自然语言问题的另一种思路是，首先定位出自然问句中的主题实体，然后从知识库中的该实体相邻节点中选出候选答案，再对候选实体进行排序得到最后的答案。
- ▶ 基于检索的知识图谱问答有两个核心问题：
 - 1) 主题实体的识别，如果主题实体定位错误，之后的候选实体很大可能也是错误的。当知识图谱较复杂时，这需要通过实体链接将文本中的实体指代与其知识图谱中真正对应的实体完成映射；
 - 2) 排序模型的效果，候选实体的得分也直接决定最终结果，排序模型可以采用规则或者深度学习的方法实现。
- ▶ 基于图谱的问答系统，利用 Freebase 知识图谱回答问题。假设一个自然语言问题可能包括的主题，在知识库中找相关的主题节点，然后从主题节点距离几跳的相关节点根据节点间的关系发现答案。
- ▶ 方法步骤:(1) 解析自然语言问句，围绕主题实体建立关于问句潜在特征的问题图;(2) 利用主题实体在 Freebase 中的邻近节点建立主题图;(3) 对齐问题图的特征和主题图的特征，找到问题与答案之间的关联;(4) 关系映射，即通过概率计算得到最可能为问题答案的关系。

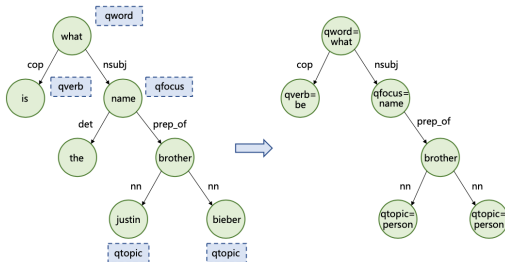
基于信息检索的知识图谱问答—1. 问题图构建

- 左图使用虚线框中带注释的问题进行依存分析，转换后的特征图(右图)仅保留有关原始问题的相关和一般信息。许多语言信息对于答案提取提供了很多信息: 疑问词 (qword) 反映了答案的类型, 例如 What/who/how; 问题焦点 (qfocus) 给出了预期答案类型的提示, 例如姓名/金钱/时间; 从问题的主要动词中提取的疑问动词 (qverb), 对答案类型也有很好的提示作用, 例如动词“演奏”之后可能会跟着乐器、电影或运动队; 问题主题 (qtopic) 则有助于找到相关的 Freebase 页面。



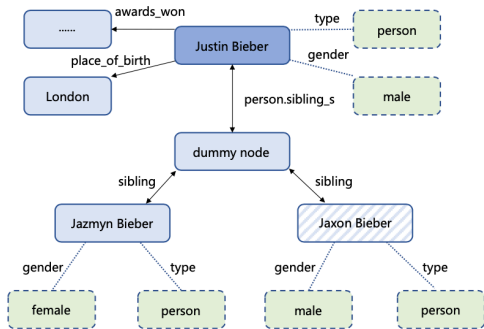
基于信息检索的知识图谱问答—1. 问题图构建

- ▶ 将问句的依存树转换为更通用的问题图，主要包含如下步骤：
 - (1) 如果一个节点被标记了一个问题特征，那么用它的问题特征替换这个节点，例如，what→qword=what;
 - (2) 如果 qtopic 节点被标记为命名实体，则将该节点替换为其命名实体形式，例如 bieber →qtopic=person;
 - (3) 删除任何作为限定词、介词或标点符号的叶节点。转换后的结果称为问题特征图，图中每个节点和关系都是这个问题的潜在特征。



基于信息检索的知识图谱问答—2. 主题图构建

- 给定一个主题，通过选择与主题节点关系在若干跳内的节点形成主题图。除了传入和传出关系之外，节点还应具有属性：描述节点属性的字符串，例如节点类型、性别或身高。以 Justin Bieber 为主题节点，通过节点间关系扩展后得到的主题图。图中实线框为节点，虚线框内为节点属性。阴影节点 Jaxon Bieber 为目标答案。



基于信息检索的知识图谱问答—3. 特征生成

- ▶ 在问题图和主题图构建完成后，需要构建人工特征用于表示问题和答案之间的关系。构建了两种特征：问题特征和候选答案特征。

1) 问题特征：在问题特征图中的每一条边 $e(s,t)$ ，提取 $s, t, s|t, s|e|t$ 作为特征。其中 s 是源节点， t 是目标节点， e 表示关系。

例如，通过边 `prep_of(qfocus=name, brother)` 可以提取以下特征：

`qfocus=name, brother, qfocus=name|brother,`
`qfocus=name|prep_of|brother。`

2) 候选答案特征：一个节点的关系和属性对区分答案非常重要，对于一个问题对应的主题图，提取每个节点的所有关系和属性作为特征。然后将问题的一个特征和候选答案的一个特征组合在一起，这样可以捕捉问题模式和答案节点之间的关系。

例如，问题-答案组合特征：`qfocus=money |`
`node_type=currency。`

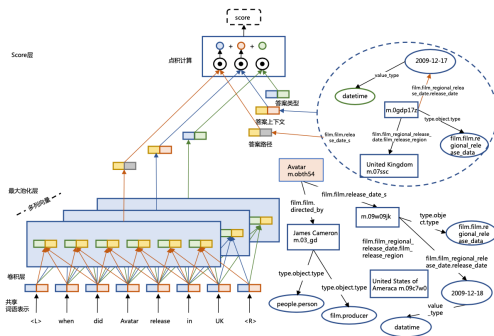
基于信息检索的知识图谱问答—4. 关系映射

- ▶ 关系映射的目标是构建知识库中的关系与自然语言单词之间的映射，发现问题所对应的关系。
- ▶ 关系映射可以形式化的定义为：给定一个包含多个单词 w 的问题 Q ，希望找出使概率 $P(R|Q)$ 最大化的关系 R 。
- ▶ 如果没有关系 R 可以使得条件概率足够大，可以采用“子关系” (sub-relation): 关系 R 是一系列子关系 $R = r = r1.r2.r3$ 的串联。
- ▶ 为了获得先验概率和条件概率，需要大量的数据进行统计。使用 ClueWeb09 语料库以及基于该语料库的实体标注语料库 FACC1。针对 Freebase 中的关系，使用包含关系的两个实体的句子进行关系学习 $P(w|R)$ 和 $P(w|r)$ 。

基于深度神经网络的知识图谱问答

基于多列卷积神经网络的知识图谱问答

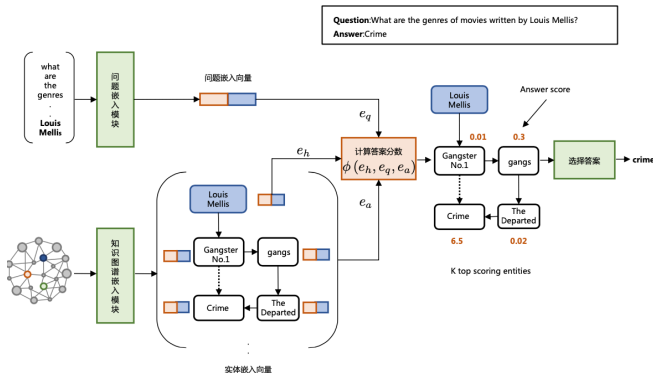
- ▶ 多列卷积神经网络 (Multi-Column Convolutional Neural Networks, MCCNN) 来进行知识图谱问答。具体来说, MCCNN 使用不同的列网络从输入问题中提取答案类型、关系和上下文信息, 同时将知识库中的实体和关系也编码为低维向量。在此基础上, 使用评分层根据问题和候选答案的表示对候选答案进行排名。



基于深度神经网络的知识图谱问答

基于知识图谱多跳问答算法 EmbedKGQA

- EmbedKGQA 使用知识图谱嵌入来回答多跳自然语言问题，并利用缺失链接预测可以用来降低知识图谱的稀疏性，提升知识图谱在多跳问答的表现。



本章小结

第十二章知识图谱

12.1 知识图谱概述

12.2 知识图谱表示与存储

12.3 知识图谱获取与构建

12.4 知识图谱推理

12.5 知识图谱问答